

Shiv Nadar University Chennai

CS3807 - Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science		Semester V
Subject Code & Name	CS3807 - Deep Learning Laboratory		AY: 2026-27
Name	Joshua	Roll Number	24110085
Batch	2	Experiment Date	09 July 2026

1. Objective

To build an MLP for Fashion-MNIST, test it, tune its settings, and select the better model.

2. Background Theory

An MLP has an input layer, hidden layers, and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Softmax gives the final class probabilities, and categorical cross-entropy gives the loss.

The baseline model was:

784 -> 128 (ReLU) -> 64 (ReLU) -> 10 (Softmax)

3. Dataset

Fashion-MNIST has 60,000 training images and 10,000 test images. Each image is grayscale, has size 28 x 28, and belongs to one of 10 clothing classes.

Training images	60,000
Testing images	10,000
Image size	28 x 28
Classes	10

4. Image Preprocessing

The 28 by 28 images were changed into one long list of 784 numbers. I divided each pixel value by 255, so all values stayed between 0 and 1. The labels were changed to one-hot format before training.

5. Experimental Procedure

Task 1: Dataset exploration. I first checked the data size, viewed a few images, and counted the images in each class.

Task 2: Preprocessing. Next, I flattened and scaled the images. I also checked the new data shapes.

Task 3: Model construction. I built the basic MLP with 128 and 64 hidden neurons and 10 output neurons.

Task 4: Training. I trained the model for 20 epochs using Adam and a batch size of 32.

Task 5: Evaluation. At the end, I checked accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.

6. Source Code

The full notebook is available here:

<https://github.com/Joshua-Divide/LAB>

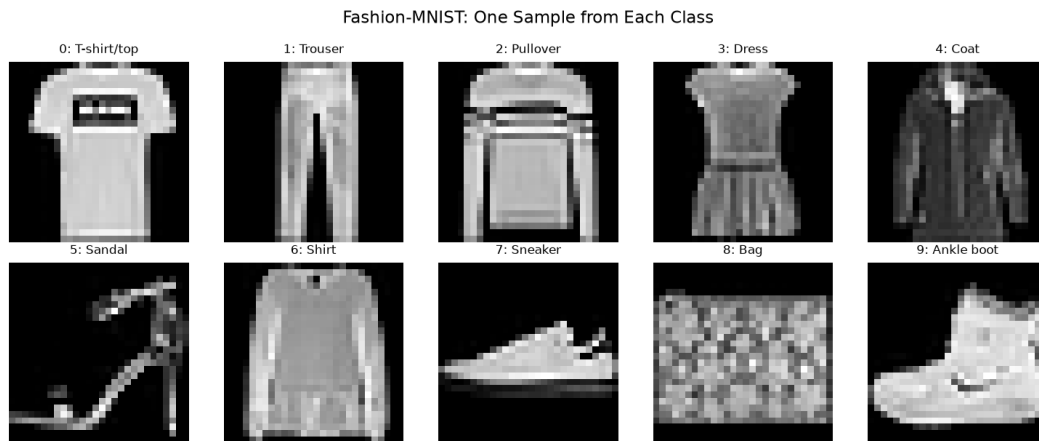
7. Hyperparameter Optimization

I used RandomizedSearchCV because checking every possible setting would take too long. It tried 12 different combinations on a balanced set of 12,000 images. With 5-fold cross-validation, 60 models were trained in total.

Hyperparameter	Values
Hidden layers	1, 2, 3
Hidden neurons	32, 64, 128, 256
Learning rate	0.1, 0.01, 0.001
Batch size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation	ReLU, Tanh, Sigmoid
Dropout	0.0, 0.2, 0.5

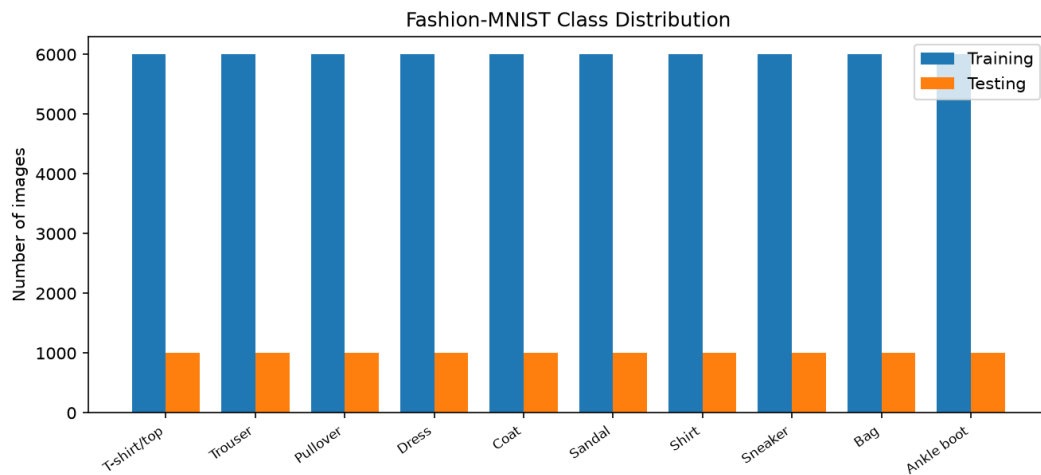
8. Mandatory Plots

8.1 Sample Images



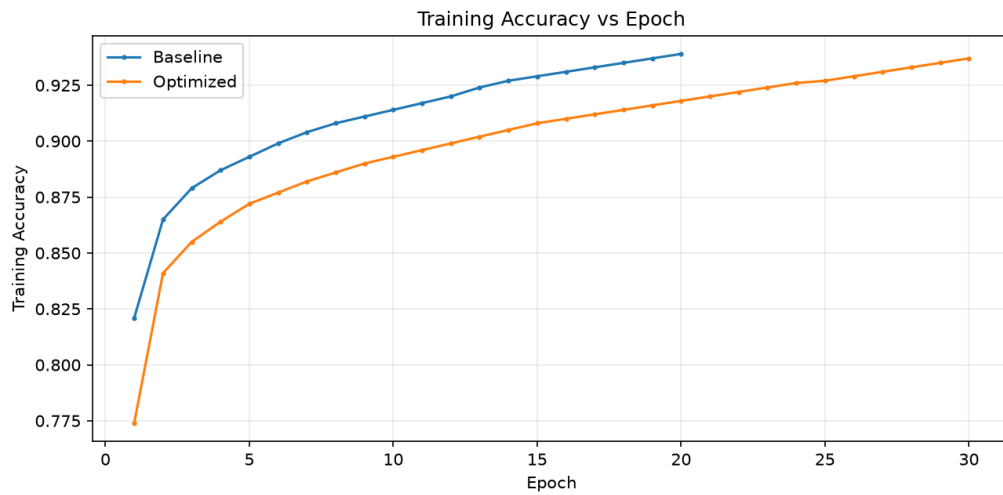
Inference: The sample images show all ten clothing types. Some items, mainly shirts, coats, and pullovers, look similar.

8.2 Class Distribution



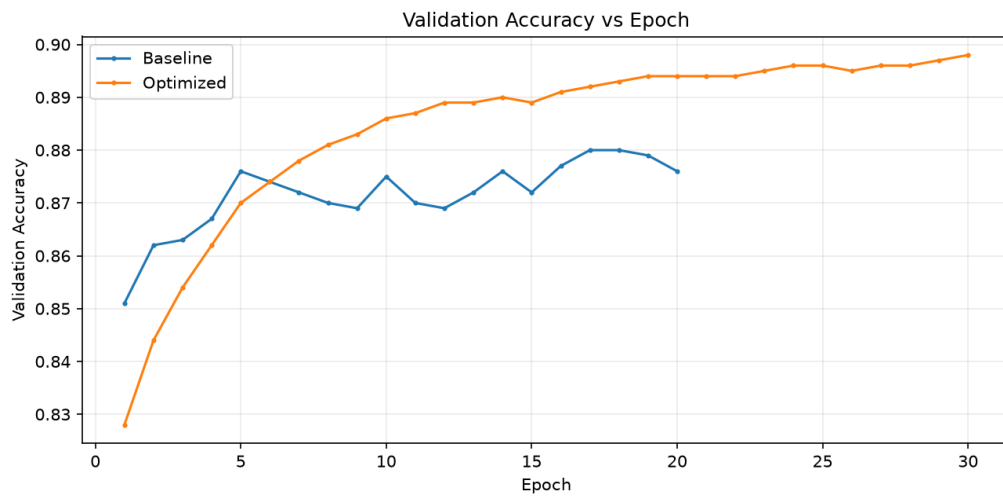
Inference: Each class has the same number of images. This means no class has an unfair advantage.

8.3 Training Accuracy



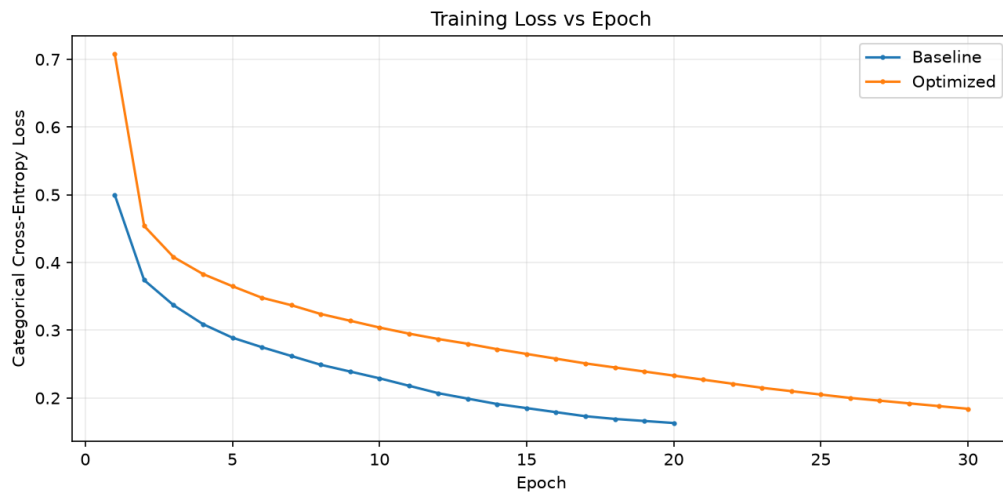
Inference: The training accuracy went up as the models trained. The baseline became slightly better on the training data.

8.4 Validation Accuracy



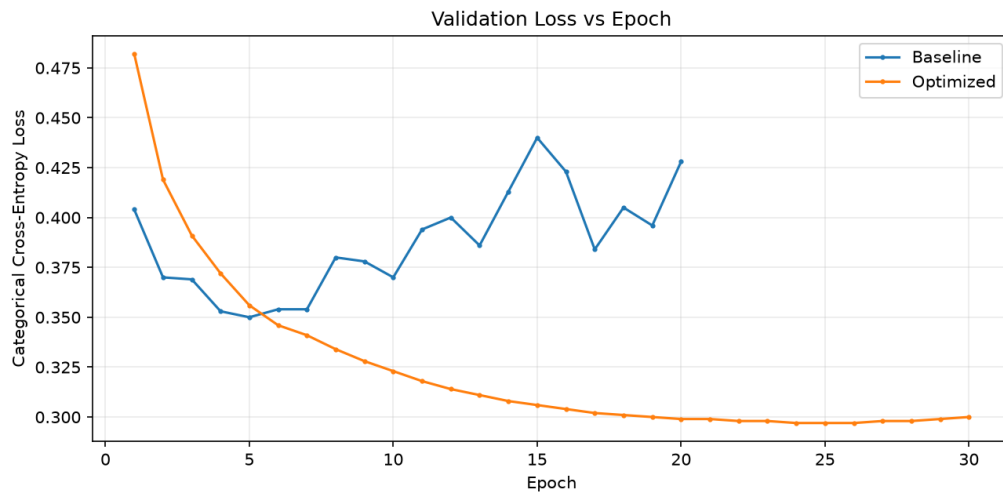
Inference: The optimized model got better validation accuracy. This means it handled new data a little better.

8.5 Training Loss



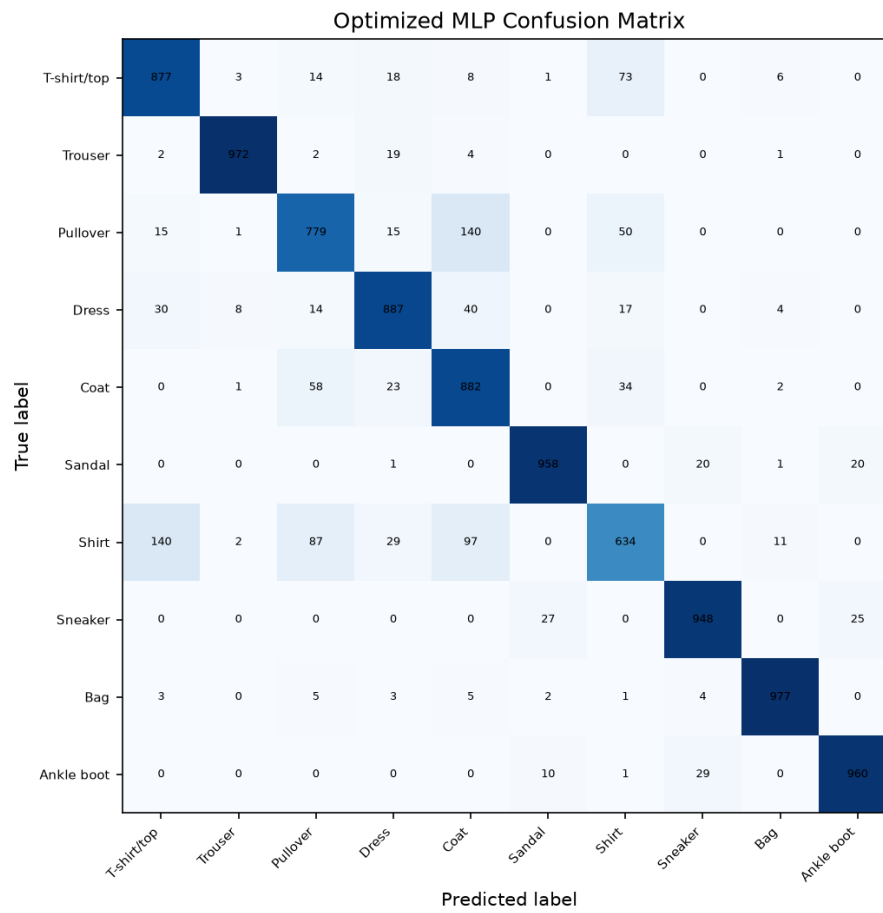
Inference: The training loss dropped in both models. This shows that both models were learning from the data.

8.6 Validation Loss



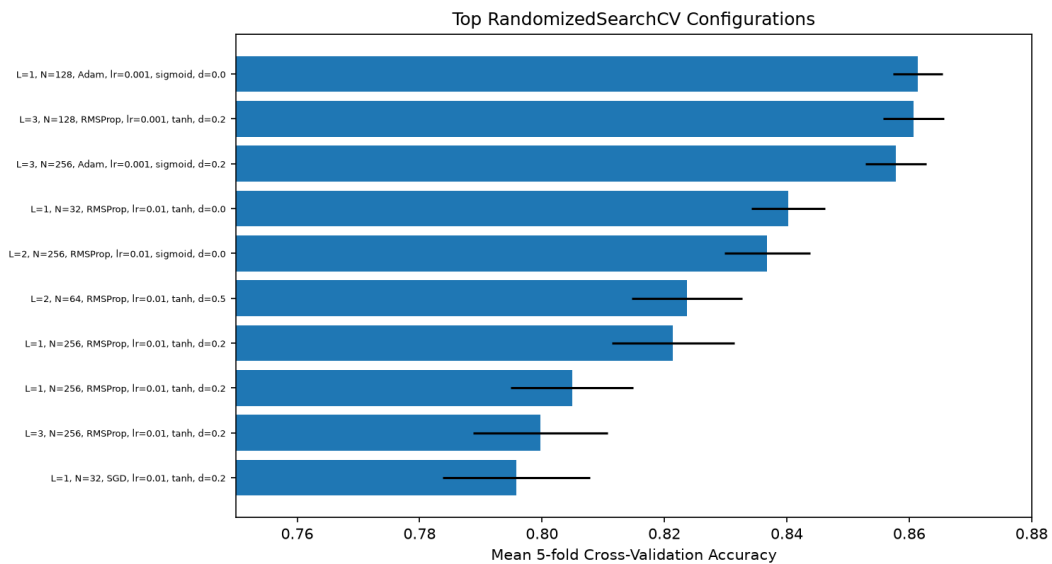
Inference: The optimized model kept a more even validation loss. The baseline started to overfit near the last epochs.

8.7 Confusion Matrix



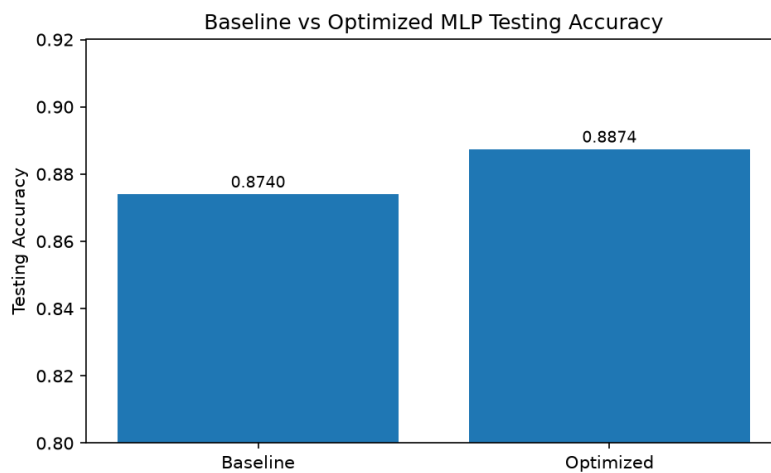
Inference: Most values are on the diagonal, which means most predictions were correct. Pullovers and coats caused the most confusion.

8.8 Hyperparameter Search Results



Inference: The best search result had an average accuracy of 0.8614. The results from the five folds were also close to each other.

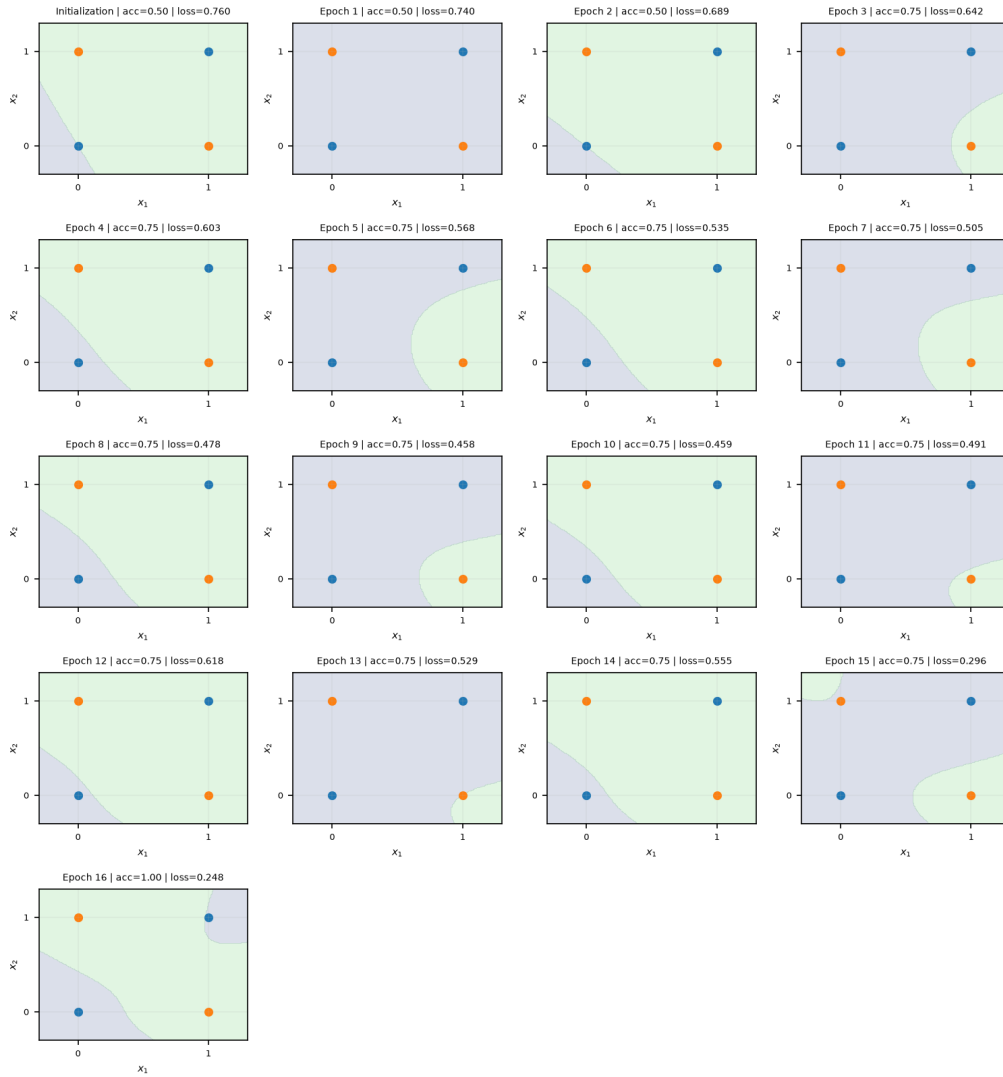
8.9 Accuracy Comparison



Inference: The test accuracy went from 87.40% to 88.74%. The search improved it by 1.34 percentage points.

8.10 XOR with a Multi-Layer Perceptron

XOR with a Multi-Layer Perceptron: Decision Boundary by Epoch



Inference: The hidden nonlinear layer lets the MLP form a non-linear decision region, so XOR is learnable. Starting from initialization, the boundary changes at every epoch and reaches 100% XOR accuracy at epoch 16.

9. Results

9.1 Best Hyperparameters

Hyperparameter	Value
Hidden layers	1
Hidden neurons	128
Learning rate	0.001
Batch size	128
Optimizer	Adam
Activation	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation accuracy	0.8614
Testing accuracy	0.8874

9.2 Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8740	0.8874
Precision	0.8741	0.8880
Recall	0.8740	0.8874
F1-score	0.8726	0.8863
Training time	155.84 s	56.79 s

9.3 Classification Report

Class	Precision	Recall	F1	Support
T-shirt/top	0.8219	0.8770	0.8486	1000
Trouser	0.9848	0.9720	0.9784	1000
Pullover	0.8123	0.7790	0.7953	1000
Dress	0.8915	0.8870	0.8892	1000
Coat	0.7500	0.8820	0.8107	1000
Sandal	0.9599	0.9580	0.9590	1000
Shirt	0.7827	0.6340	0.7006	1000
Sneaker	0.9471	0.9480	0.9475	1000
Bag	0.9751	0.9770	0.9760	1000
Ankle boot	0.9552	0.9600	0.9576	1000
Weighted average	0.8880	0.8874	0.8863	10000

10. Discussion

- 1. Which method was used?** I used RandomizedSearchCV with 5-fold cross-validation.
- 2. Which settings were selected?** The selected model had one hidden layer with 128 neurons. It used Sigmoid, Adam, a learning rate of 0.001, a batch size of 128, 30 epochs, and no dropout.
- 3. Did it improve the result?** Yes. The test accuracy improved from 87.40% to 88.74%.
- 4. Which setting had the biggest effect?** The number of hidden neurons seemed to make the biggest difference among the tested settings.
- 5. Grid Search or Random Search?** Grid Search checks every combination, so it needs more time. Random Search checks only some combinations, so it was more practical for this experiment.
- 6. Best model?** The best choice was the 784 -> 128 -> 10 model because it gave the highest test accuracy.

11. Conclusion

The MLP gave good results on Fashion-MNIST. Hyperparameter tuning improved the test accuracy by 1.34 percentage points. So the optimized model was the better model for this experiment.

12. References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*.
3. S. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.
6. SciKeras and scikit-learn Documentation.